

# 代码生成与寄存器分配完整模拟卷 05（题目与完整解析）

## 卷面说明

考试时间：120 分钟

总分：100 分

说明：本卷独立覆盖指令选择、指令代价、基本块/流图、活跃性、DAG 优化、描述符和 getReg。

目标机语义及代价均在题面中给出。

## A 卷题目

### 一、指令选择与目标代码代价（20 分）

目标机有寄存器 R0、R1，指令及代价如下：

|      |        |            |      |
|------|--------|------------|------|
| LD   | R, x   | ; R=x      | 代价 2 |
| ST   | x, R   | ; x=R      | 代价 2 |
| ADD  | Rd, Rs | ; Rd=Rd+Rs | 代价 1 |
| ADDI | R, k   | ; R=R+k    | 代价 1 |
| INC  | x      | ; x=x+1    | 代价 1 |

1. 为三地址语句  $x=y+z$  生成目标代码并计算总代价。 2. 用通用 LD/ADDI/ST 序列实现  $a=a+1$ ，计算代价。 3. 再用 INC a 实现同一语句，计算节省的代价。 4. 说明同一 IR 为什么可能对应多个目标指令序列。 5. 写出代码生成器在指令选择时必须考虑的三个因素。

### 二、基本块划分、流图与循环（20 分）

给定三地址指令：

- (1)  $b = 1$
- (2)  $n = 10$
- (3)  $d = l + n$
- (4)  $c = 2$
- (5)  $t1 = 4 * a$
- (6)  $t2 = l + n$
- (7)  $c = c + b$
- (8) if  $c < n$  goto (6)
- (9)  $a = a + b$
- (10) if  $a < n$  goto (4)
- (11)  $c = a - b$

1. 写出首指令判定规则。 2. 列出全部首指令。 3. 划分基本块。 4. 写出完整 CFG 边集合。 5. 写出回边及对应自然循环。

### 三、活跃性与下一次使用（20 分）

基本块：

- (1)  $t1 = a - b$
- (2)  $t2 = t1 + c$
- (3)  $t3 = t2 + d$
- (4)  $a = t3$

块出口处：

LiveOut={a, b, c, d}  
t1、t2、t3 不活跃且无后续使用

1. 写出对语句  $x=y \text{ op } z$  反向扫描的更新规则。 2. 计算每条语句之前的活跃变量集合。 3. 写出每个源操作数在块内的下一次使用位置。 4. 指出每条语句的结果在何时变为不活跃。 5. 说明这些信息怎样帮助 `getReg` 选择可覆盖寄存器。

#### 四、基本块 DAG、数组写入与死代码（20 分）

给定基本块：

```
(1) x = a[i]
(2) y = a[i]
(3) a[j] = t
(4) z = a[i]
(5) u = b + 1
(6) v = b + 1
(7) w = v * 1
```

已知无法证明  $i!=j$ ，块出口活跃变量为：

`LiveOut={x, z, u}`

1. 指出可以合并的公共子表达式。 2. 说明数组写入 (3) 为什么会杀死此前的 `a[i]` 结果。 3. 对 (7) 做代数化简。 4. 删除死代码后写出最终基本块。 5. 区分死代码和不可达代码。

#### 五、描述符、`getReg` 与两寄存器代码生成（20 分）

目标机只有 `R0`、`R1`，采用两地址指令：

```
LD  R, x      ; R=x
ST  x, R      ; x=R
ADD Rd, Rs    ; Rd=Rd+Rs
SUB Rd, Rs    ; Rd=Rd-Rs
```

三地址码：

```
t1 = a - b
t2 = a - c
t3 = t1 + t2
d  = t3 + t2
```

1. 写出寄存器描述符和地址描述符的定义。 2. 给出一组完整目标代码，允许把临时变量溢出到内存。 3. 逐条写出 `R0`、`R1` 中保存的值。 4. 指出哪一步发生 `spill`，哪一步发生 `reload`。 5. 写出结束时 `d` 的地址描述符。

## B 按考试标准重写答案

### 一、指令选择与目标代码代价答案

#### 1. `x=y+z`

目标机是寄存器-寄存器加法，因此先把两个操作数装入寄存器，再相加并写回：

```
LD R0, y      ; 代价 2
LD R1, z      ; 代价 2
ADD R0, R1    ; 代价 1, R0=y+z
ST x, R0      ; 代价 2
```

总代价：

$$2 + 2 + 1 + 2 = 7$$

#### 2. 用通用序列实现 `a=a+1`

```
LD R0, a      ; 代价 2
ADDI R0, 1    ; 代价 1
ST a, R0      ; 代价 2
```

总代价：

$$2 + 1 + 2 = 5$$

#### 3. 用 `INC a`

```
INC a          ; 代价 1
```

节省代价：

$$5 - 1 = 4$$

#### 4. 同一 IR 对应多个目标序列的原因

同一条 IR 只描述语义，不指定机器指令形态。目标机可能有多种实现方式：

| IR 语义 | 实现方式           |
|-------|----------------|
| a=a+1 | LD/ADDI/ST     |
| a=a+1 | INC a          |
| x=y+z | 不同寄存器分配、不同装载顺序 |

只要最终可观察结果一致，代码生成器可以选择不同寻址方式、寄存器和专用指令。

#### 5. 指令选择考虑因素

| 因素     | 说明                    |
|--------|-----------------------|
| 目标 ISA | 指令语义、寻址方式、是否有立即数或复合指令 |
| 代码代价   | 指令条数、执行周期、访存次数        |
| 寄存器压力  | 当前可用寄存器数量，是否会造成 spill |
| IR 信息  | IR 是否保留类型、内存访问、常量等信息  |
| 后续优化   | 是否便于调度、寄存器分配和流水线执行    |

### 二、基本块划分、流图与循环答案

#### 1. 首指令判定规则

首指令有三类：

- 1. 整个程序或指令序列的第一条指令。
- 2. 任意跳转指令的目标指令。
- 3. 紧跟在跳转指令之后的指令。

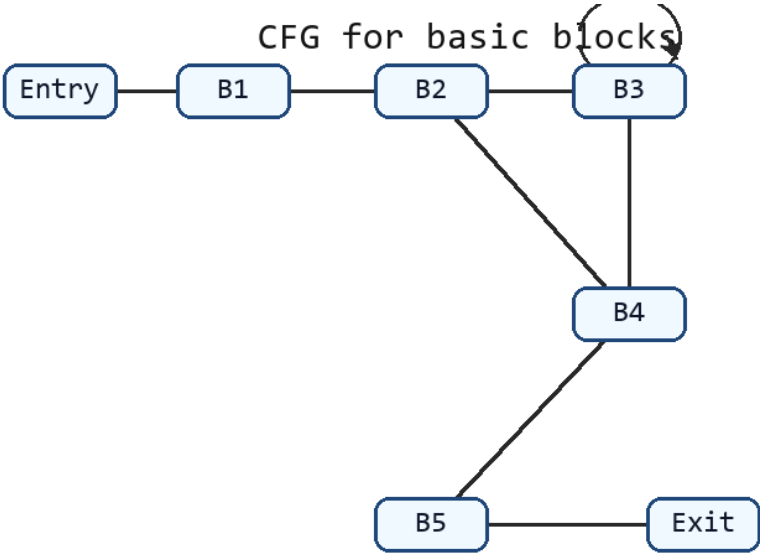
2. 全部首指令

| 首指令  | 原因             |
|------|----------------|
| (1)  | 第一条指令          |
| (4)  | 指令 (10) 的跳转目标  |
| (6)  | 指令 (8) 的跳转目标   |
| (9)  | 紧跟在条件跳转 (8) 后  |
| (11) | 紧跟在条件跳转 (10) 后 |

3. 基本块划分

| 基本块 | 指令范围       |
|-----|------------|
| B1  | (1) – (3)  |
| B2  | (4) – (5)  |
| B3  | (6) – (8)  |
| B4  | (9) – (10) |
| B5  | (11)       |

4. CFG 边集合



基本块 CFG

| 边          | 原因                 |
|------------|--------------------|
| B1 -> B2   | B1 顺序落到 B2         |
| B2 -> B3   | B2 顺序落到 B3         |
| B3 -> B3   | (8) 条件为真，跳到 (6)    |
| B3 -> B4   | (8) 条件为假，顺序到 (9)   |
| B4 -> B2   | (10) 条件为真，跳到 (4)   |
| B4 -> B5   | (10) 条件为假，顺序到 (11) |
| B5 -> Exit | 程序片段结束             |

5. 回边和自然循环

回边定义：边  $n \rightarrow h$  中，若  $h$  支配  $n$ ，则该边为回边。

| 回边                  | 判断       | 自然循环         |
|---------------------|----------|--------------|
| $B3 \rightarrow B3$ | 自身支配自身   | {B3}         |
| $B4 \rightarrow B2$ | B2 支配 B4 | {B2, B3, B4} |

B4 到 B2 的自然循环从回边尾 B4 反向找前驱，直到回边头 B2，得到 B4、B3、B2。

### 三、活跃性与下一次使用答案

#### 1. 反向扫描规则

对语句：

$x = y \text{ op } z$

从基本块末尾向前扫描：

1. 先记录该语句处  $x$ 、 $y$ 、 $z$  当前的 live 和 next-use 信息。
2. 因为本语句重新定义  $x$ ，所以把  $x$  设为不活跃，next-use 设为空。
3. 因为本语句使用  $y$ 、 $z$ ，所以把  $y$ 、 $z$  设为活跃，next-use 设为当前语句编号。

#### 2. 每条语句之前的活跃变量集合

出口：

LiveOut = {a, b, c, d}

反向计算：

| 位置        | 活跃变量          |
|-----------|---------------|
| 语句 (4) 之后 | {a, b, c, d}  |
| 语句 (4) 之前 | {b, c, d, t3} |
| 语句 (3) 之前 | {b, c, d, t2} |
| 语句 (2) 之前 | {b, c, d, t1} |
| 语句 (1) 之前 | {a, b, c, d}  |

以语句 (4) 为例：

$a = t3$   
 $\text{before} = (\text{after} - \{a\}) \cup \{t3\}$   
 $\quad = \{b, c, d, t3\}$

#### 3. 源操作数下一次使用

| 位置       | 源操作数  | 下一次使用 |
|----------|-------|-------|
| 语句 (1) 前 | a, b  | (1)   |
| 语句 (2) 前 | t1, c | (2)   |
| 语句 (3) 前 | t2, d | (3)   |
| 语句 (4) 前 | t3    | (4)   |

#### 4. 结果何时变为不活跃

| 结果 | 产生位置 | 最后一次使用 | 之后状态        |
|----|------|--------|-------------|
| t1 | (1)  | (2)    | 语句 (2) 后不活跃 |
| t2 | (2)  | (3)    | 语句 (3) 后不活跃 |
| t3 | (3)  | (4)    | 语句 (4) 后不活跃 |

| 结果  | 产生位置 | 最后一次使用 | 之后状态 |
|-----|------|--------|------|
| 新 a | (4)  | 块外仍活跃  | 出口活跃 |

## 5. 对 getReg 的帮助

getReg 选择寄存器时优先覆盖“后面不再使用”的值：

| 情况             | 策略                |
|----------------|-------------------|
| 某寄存器中的值之后不活跃   | 可直接覆盖             |
| 值之后活跃但内存已有最新副本 | 可覆盖寄存器副本          |
| 所有值都活跃且无内存副本   | 选择下一次使用最晚的值 spill |

因此活跃性和下一次使用信息能减少不必要的 ST 和 LD。

## 四、基本块 DAG、数组写入与死代码答案

### 1. 可合并的公共子表达式

| 表达式                     | 可否合并 | 原因                |
|-------------------------|------|-------------------|
| (1) a[i] 与 (2) a[i]     | 可以   | 两者之间没有写 a，也没有修改 i |
| (5) b+1 与 (6) b+1       | 可以   | 两者操作数相同，且 b 未变    |
| (1)/(2) a[i] 与 (4) a[i] | 不可以  | 中间有 (3) a[j]=t    |

### 2. 数组写入为什么 kill 先前读取

题目明确无法证明  $i \neq j$ 。因此  $a[j]=t$  可能写的就是  $a[i]$ ：

若  $i == j$ ，则  $a[j]$  与  $a[i]$  是同一位置。

所以 (3) 必须杀死此前关于数组 a 的读取结果，(4) 的  $z=a[i]$  需要重新读取。

### 3. 代数化简

$$w = v * 1$$

可化简为：

$$w = v$$

### 4. 删除死代码后的最终基本块

出口活跃变量为：

$$\text{LiveOut} = \{x, z, u\}$$

y、v、w 在出口不活跃，且它们的值不影响活跃变量。因此最终保留：

- (1)  $x = a[i]$
- (3)  $a[j] = t$
- (4)  $z = a[i]$
- (5)  $u = b + 1$

说明： $a[j]=t$  即使没有变量名出现在 LiveOut 中也不能删，因为数组写入是内存副作用。

### 5. 死代码与不可达代码

| 类型  | 含义                           |
|-----|------------------------------|
| 死代码 | 会被执行，但计算结果以后不再使用，且删除不改变可观察行为 |

| 类型    | 含义                     |
|-------|------------------------|
| 不可达代码 | 从 CFG 入口没有路径到达，运行时不会执行 |

## 五、描述符、getReg 与两寄存器代码生成答案

### 1. 描述符定义

| 描述符               | 定义                             |
|-------------------|--------------------------------|
| 寄存器描述符 RegDesc(R) | 记录寄存器 R 当前保存哪些变量或临时变量的最新值      |
| 地址描述符 AddrDesc(x) | 记录变量 x 的最新值当前在哪些位置：内存、寄存器或两者都有 |

### 2. 一组完整目标代码

只有两个寄存器，计算 t2 时需要覆盖保存 t1 的寄存器，所以先 spill t1 到内存，后面再 reload。

```

LD R0, a
LD R1, b
SUB R0, R1      ; R0 = t1
ST t1, R0       ; spill t1

LD R0, a
LD R1, c
SUB R0, R1      ; R0 = t2

LD R1, t1       ; reload t1
ADD R1, R0      ; R1 = t3 = t1 + t2
ADD R1, R0      ; R1 = d = t3 + t2
ST d, R1

```

### 3. 寄存器内容变化

| 步骤 | 指令         | R0   | R1   |
|----|------------|------|------|
| 初始 | -          | {}   | {}   |
| 1  | LD R0, a   | {a}  | {}   |
| 2  | LD R1, b   | {a}  | {b}  |
| 3  | SUB R0, R1 | {t1} | {b}  |
| 4  | ST t1, R0  | {t1} | {b}  |
| 5  | LD R0, a   | {a}  | {b}  |
| 6  | LD R1, c   | {a}  | {c}  |
| 7  | SUB R0, R1 | {t2} | {c}  |
| 8  | LD R1, t1  | {t2} | {t1} |
| 9  | ADD R1, R0 | {t2} | {t3} |
| 10 | ADD R1, R0 | {t2} | {d}  |
| 11 | ST d, R1   | {t2} | {d}  |

### 4. spill 和 reload

| 类型     | 指令        | 原因                        |
|--------|-----------|---------------------------|
| spill  | ST t1, R0 | 后面还要用 t1，但 R0 要被重新用于计算 t2 |
| reload | LD R1, t1 | 计算 t3 时需要重新取回 t1          |

### 5. 结束时 `d` 的地址描述符

最后执行了：

```
ST d, R1
```

因此 d 的最新值既在 R1，也在内存 d 中：

$\text{AddrDesc}(d) = \{R1, \text{内存位置 } d\}$